Ashwath Raj M | AP25110010712 | Core CSE A

GITHUB LINK: https://github.com/Ashwath-Raj/CSE-204-DAA-assignments-

1. Minimum Spanning Tree Using Kruskal's Algorithm

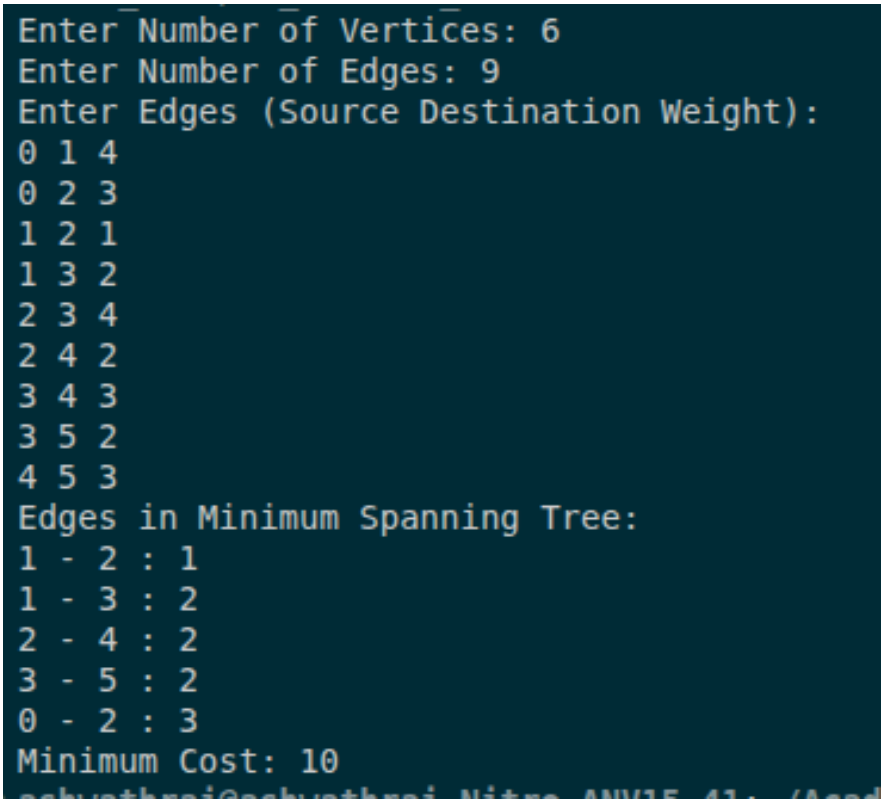
Problem

Implement a Minimum Spanning Tree for a weighted undirected graph using Kruskal's Algorithm.

Method

Kruskal's Algorithm sorts all graph edges in increasing order of weight and repeatedly selects the smallest edge that does not form a cycle with the already selected edges. A Disjoint Set Union structure is used to detect cycles. The process continues until V - 1 edges are selected.

Output:



Code:

```
/*
Program Name: Minimum Spanning Tree Using Kruskal's Algorithm
Program Description: Takes a weighted undirected graph as input and finds
                     its Minimum Spanning Tree using Kruskal's Algorithm
Program Creation Time: Sep 20th
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<algorithm>
using namespace std;

struct Edge{
    int u;
    int v;
    int weight;
};

bool compare(Edge a,Edge b){
    return a.weight<b.weight;
}

int findParent(int parent[],int node){
    if(parent[node]==node)
        return node;

    return parent[node]=findParent(parent,parent[node]);
}
void unionSet(int parent[],int rank[],int u,int v){
```

```
    u=findParent(parent,u);
    v=findParent(parent,v);

    if(u!=v){
        if(rank[u]<rank[v])
            parent[u]=v;
        else if(rank[u]>rank[v])
            parent[v]=u;
        else{
            parent[v]=u;
            rank[u]++;
        }
    }
}

void kruskal(Edge edges[],int V,int E){
    sort(edges,edges+E,compare);

    int parent[V],rank[V];

    for(int i=0;i<V;i++){
        parent[i]=i;
        rank[i]=0;
    }

    int count=0,total=0;

    cout<<"Edges in Minimum Spanning Tree:"<<endl;

    for(int i=0;i<E && count<V-1;i++){
        int u=edges[i].u;
        int v=edges[i].v;

        if(findParent(parent,u)!=findParent(parent,v)){
            cout<<u<<" - "<<v<<" : "<<edges[i].weight<<endl;
            total+=edges[i].weight;
            unionSet(parent,rank,u,v);
            count++;
        }
    }

    cout<<"Minimum Cost: "<<total<<endl;
}

int main(void){
    int V,E;

    cout<<"Enter Number of Vertices: ";
    cin>>V;

    cout<<"Enter Number of Edges: ";
    cin>>E;

    Edge edges[E];

    cout<<"Enter Edges (Source Destination Weight):"<<endl;
    for(int i=0;i<E;i++)
        cin>>edges[i].u>>edges[i].v>>edges[i].weight;

    kruskal(edges,V,E);

    return 0;
}
```

Complexity

Time Complexity: O(E log E)
Space Complexity: O(V + E)

The edges are first sorted in O(E log E) time. Each edge is then processed using the Disjoint Set Union structure, with near constant amortized cost for find and union operations. Therefore, sorting dominates the overall complexity.

2. Minimum Spanning Tree Using Prim's Algorithm

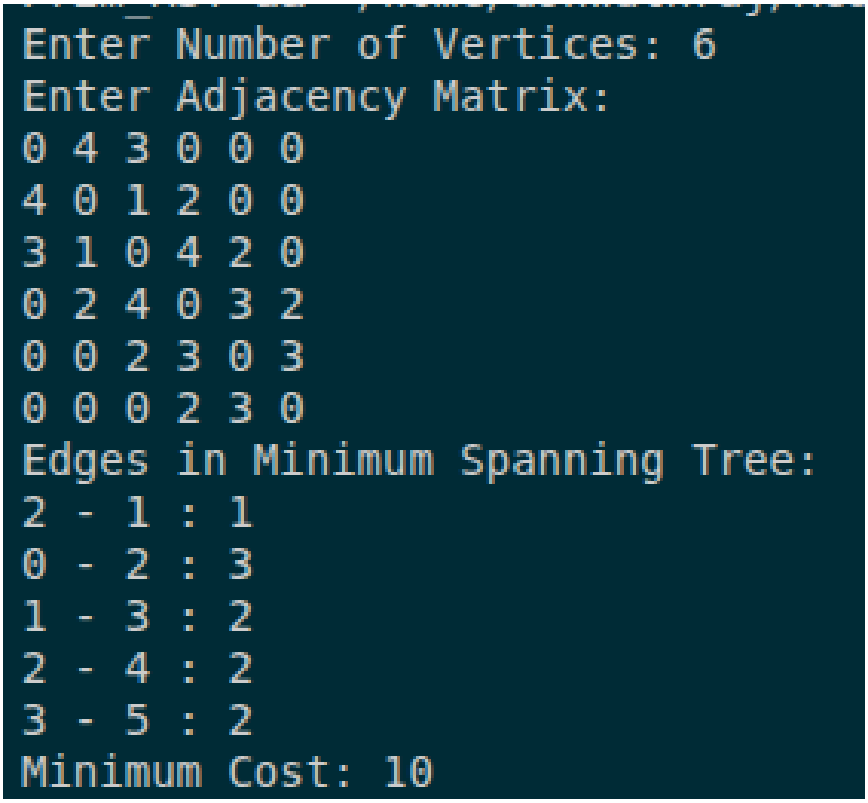
Problem

Implement a Minimum Spanning Tree for a weighted undirected graph using Prim's Algorithm.

Method

Prim's Algorithm starts from vertex 0 and grows the Minimum Spanning Tree one vertex at a time. At every step, the minimum weight edge connecting a visited vertex to an unvisited vertex is selected. The process continues until all vertices are included in the tree.

Output:



Code:

```
/*
Program Name: Minimum Spanning Tree Using Prim's Algorithm
Program Description: Takes a weighted undirected graph as input and finds
                     its Minimum Spanning Tree using Prim's Algorithm
Program Creation Time: Sep 20th
Program Creator: Ashwath Raj | AP25110010712
*/

#include<iostream>
#include<climits>
using namespace std;

int findMin(int key[],bool visited[],int V){
    int minValue=INT_MAX;
    int index=-1;

    for(int i=0;i<V;i++){
        if(!visited[i] && key[i]<minValue){
            minValue=key[i];
            index=i;
        }
    }

    return index;
}

void prim(int graph[][10],int V){
    int parent[V],key[V];
    bool visited[V]={false};

    for(int i=0;i<V;i++){
        key[i]=INT_MAX;
        parent[i]=-1;
    }
```

```
    key[0]=0;

    for(int i=0;i<V-1;i++){
        int u=findMin(key,visited,V);

        visited[u]=true;

        for(int v=0;v<V;v++){
            if(graph[u][v] && !visited[v] && graph[u][v]<key[v]){
                parent[v]=u;
                key[v]=graph[u][v];
            }
        }
    }

    int total=0;

    cout<<"Edges in Minimum Spanning Tree:"<<endl;

    for(int i=1;i<V;i++){
        cout<<parent[i]<<" - "<<i<<" : "<<graph[i][parent[i]]<<endl;
        total+=graph[i][parent[i]];
    }

    cout<<"Minimum Cost: "<<total<<endl;
}

int main(void){
    int V;

    cout<<"Enter Number of Vertices: ";
    cin>>V;

    int graph[10][10]={0};

    cout<<"Enter Adjacency Matrix:"<<endl;
    for(int i=0;i<V;i++)
        for(int j=0;j<V;j++)
            cin>>graph[i][j];

    prim(graph,V);

    return 0;
}
```

Complexity

Time Complexity: $O(V^2)$
Space Complexity: $O(V^2)$

For every vertex, the algorithm scans the key array to find the minimum unvisited vertex. It then scans the corresponding adjacency matrix row to update the minimum connecting edges. With an adjacency matrix, the overall time complexity is $O(V^2)$.